

WYKŁAD 9

Bezpieczeństwo, certyfikacja
i wdrażanie systemów nawigacji humanoidów

dr inż. Mateusz Pomianek



Normy ISO / IEC

Runtime monitoring

Testing / Validation

Wdrożenie / DevOps

Plan wykładu

Sekcja 1

Normy i standardy: ISO 13482, IEC 61508, EN ISO 10218 - wymagania formalne dla robotów usługowych

Sekcja 2

Bezpieczeństwo w czasie wykonania: runtime monitors, fail-safe, conformal prediction, hardware e-stop

Sekcja 3

Testowanie i walidacja: HIL/SIL/PIL, scenariusze testowe, benchmarki, metryki jakości nawigacji

Sekcja 4

Wdrażanie i utrzymanie: architektura floty, OTA updates, monitoring operacyjny, etyka i perspektywy

01

SEKCJA

Normy i standardy bezpieczeństwa

ISO 13482, IEC 61508, poziomy SIL/PLe i wymagania formalne



Dlaczego normy bezpieczeństwa są obowiązkowe?

- Robot humanoidalny poruszający się wśród ludzi może zadać uraz fizyczny lub psychiczny; odpowiedzialność prawna producenta
- **Normy zmniejszają ryzyko:** definiują systematyczny proces identyfikacji zagrożeń (HAZOP, FMEA) i ich redukcji
- **ISO 13482:2014:** norma dedykowana robotom usługowym (personal care robots); obejmuje wymogi bezpieczeństwa fizycznego i funkcjonalnego
- **IEC 61508:** bezpieczeństwo funkcjonalne systemów elektrycznych i elektronicznych; definiuje poziomy SIL (Safety Integrity Level)
- **EN ISO 10218-1/2:** bezpieczeństwo robotów przemysłowych; stosowany gdy humanoid pracuje wspólnie z operatorem
- **CE Marking (dyrektywa maszynowa 2006/42/WE):** warunek konieczny do sprzedaży robota w UE
- **Brak certyfikatu = zakaz używania** w przestrzeniach publicznych i szpitalach w Polsce i UE

Poziomy nienaruszalności bezpieczeństwa SIL i PLe

SIL: Safety Integrity Level (IEC 61508)

- **SIL 1:** prawdopodobieństwo awarii niebezpiecznej 10^{-5} - $10^{-6}/h$ (np. robot w zamkniętej strefie)
- **SIL 2:** 10^{-6} - $10^{-7}/h$; wymagany dla robotów współpracujących z operatorem
- **SIL 3:** 10^{-7} - $10^{-8}/h$; roboty medyczne i w infrastrukturze krytycznej
- **SIL 4:** $< 10^{-8}/h$; lotnictwo, energetyka jądrowa - praktycznie nieosiągalny w robotyce mobilnej
- **Wyznaczyć SIL** musi jednostka notyfikowana; wymaga analizy FMEA i FTA

PLe: Performance Level (EN ISO 13849)

- **PL a-e:** od najniższego do najwyższego poziomu niezawodności funkcji bezpieczeństwa
- **PLe:** MTTFd > 100 lat; DC $\geq 99\%$; wymagany dla e-stop i watchdog-a robota mobilnego
- **PLd:** MTTFd 30-100 lat; DC $\geq 90\%$; typowy dla ograniczników prędkości i torque limits
- **Walidacja PLe:** testy starzeniowe, analiza FMEA, audyt przez TUV lub notyfikowana jednostkę
- **PLe często wymaga redundancji:** dwa niezależne czujniki kontaktu lub dwa CPU

Analiza ryzyka: HAZOP i FMEA dla humanoida

- **HAZOP (Hazard and Operability Study):** systematyczne przegląd wszystkich węzłów systemu pod kątem odchyień od normalnej pracy
- **Przykład HAZOP:** "robot nie wykryje pieszego" -> zagrożenie: kolizja -> przyczyna: zaślepiiony LiDAR -> bariera: redundantna kamera
- **FMEA (Failure Mode and Effects Analysis):** tabelaryczna analiza trybów awarii każdego komponentu i ich skutków
- **Parametry FMEA:** S - powaga skutku (1-10), O - częstotliwość wystąpienia (1-10), D - wykrywalność (1-10)
- **RPN = S × O × D;** komponenty z RPN > 100 wymagają natychmiastowej modyfikacji lub redundancji
- **Dla humanoida krytyczne komponenty:** IMU, LiDAR, system e-stop, aktuatory bioder i kolan
- **Wymagane dla CE Marking:** pełna dokumentacja analizy ryzyka przed pierwszym wdrożeniem publicznym

Wymagania ISO 13482 dla robota nawigującego wśród ludzi

Ograniczenie prędkości

Max 0,5 m/s w trybie współpracy;
max 1,5 m/s w strefie wydzielonej.
Nadzór sprzętowy.

Sily kontaktowe

Siła nacisku < 50 N na dowolną część ciała; < 35 N dla głowy i szyi. Czujnik F/T obowiązkowy.

Zatrzymanie awaryjne

E-stop: zatrzymanie w < 500 ms od wciśnięcia przycisku;
możliwość ręcznego zatrzymania przez operatora.

Detekcja człowieka

Robot musi wykryć osobę w strefie bezpieczeństwa 1 m i zareagować w < 200 ms.

Instrukcje i oznakowanie

Podręcznik użytkownika, etykiety ostrzegawcze, sygnały dźwiękowe i świetlne dla każdego trybu.

Walidacja eksperymentalna

Min. 1000 h testów w środowiskach reprezentatywnych przed certyfikatem; log incydentów.

02

SEKCJA

Bezpieczeństwo w czasie wykonania

Runtime monitors, fail-safe, conformal prediction, e-stop



Architektura bezpieczeństwa runtime: warstwy nadzoru

Warstwa 1: sprzęt

PLc hardware e-stop; wylamywacze momentow;
enkodery redundantne; zasilanie awaryjne

Warstwa 2: RTOS watchdog

Dedykowany MCU 1 kHz; nadzoruje komunikacje z
aktuatorami; resetuje przy braku heartbeat

Warstwa 3: safety monitor

Node ROS 2 lifecycle; sprawdza ZMP, predkosc, strefe
bezpieczna; veto dla planner/controller

Warstwa 4: runtime verif.

Conformal prediction lub formalne monitory STL/MTL;
certyfikuje ze $P(\text{naruszenie}) < \text{delta}$ w trybie online

Warstwa 5: operator HMI

Dashboard z alertami; tele-operacja awaryjna; przycisk
stop zdalny przez WiFi / 4G

Kluczowe cechy

- Każda warstwa działa niezależnie
- Awaria wyższej -> niższe przejma
- Latencja: warstwy 1-2 < 1 ms
- Warstwy 3-4: < 10 ms
- Warstwa 5: akceptowalne > 100 ms
- Testy redundancji co 24h w deploymentcie

Runtime monitor: co sprawdzać i kiedy

- **Prędkość liniowa i kątowa:** $|v| \leq v_max$, $|\omega| \leq \omega_max$ - sprzętowe zabezpieczenie momentów przy przekroczeniu
- **Margines ZMP:** gdy ZMP wychodzi za wielobok podparcia o > 1 cm przez > 3 cykle (15 ms) -> zmniejsz prędkość
- **Detekcja upadku:** $|\text{roll}| > 45^\circ$ lub $|\text{pitch}| > 45^\circ$ lub akcelerometr $|a| > 3g$ przez > 50 ms -> postura awaryjna
- **Strefa bliskości:** wykryty człowiek < 0.5 m -> stop natychmiastowy; < 1.0 m -> zwolnij do 0.3 m/s
- **Temperatura aktuatorów:** $> 80^\circ\text{C}$ -> ogranicz moment do 50%; $> 90^\circ\text{C}$ -> zatrzymaj i wentyluj
- **Utrata połączenia sensora:** brak wiadomości IMU przez > 50 ms -> tryb degradacji (dead reckoning + emergency stop)
- **Innowacja filtra EKF:** $NIS > \chi^2_{.95}$ przez > 5 kroków -> reset lokalizacji i stop w bezpiecznej strefie

Safety Monitor: node ROS 2 (szkielet w Python)

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Bool

class SafetyMonitor(Node):
    LIMITS = dict(v_max=0.8, omega_max=1.2, temp_warn=80,
                  zmp_margin=0.01, ped_stop=0.5, ped_slow=1.0)
    def __init__(self):
        super().__init__("safety_monitor")
        self.ok_pub = self.create_publisher(Bool, "/safety_ok", 1)
        self.create_timer(0.01, self.check) # 100 Hz
        self.state = {} # wypełniane przez subskrybentów
    def check(self):
        s = self.state
        ok = (
            abs(s.get("v",0)) <= self.LIMITS["v_max"] and
            s.get("zmp_margin",99) >= self.LIMITS["zmp_margin"] and
            s.get("ped_dist",99) >= self.LIMITS["ped_stop"] and
            s.get("temp_max",0) <= self.LIMITS["temp_warn"]
        )
        self.ok_pub.publish(Bool(data=bool(ok)))
```

Opis: /safety_ok = False -> controller manager wstrzymuje komendy i inicjuje graceful stop

Conformal Prediction dla certyfikacji runtime

- **Conformal Prediction (CP):** formalna metoda statystyczna gwarantująca $P(\text{naruszenie}) \leq \alpha$ na danych kalibracyjnych
- **Idea:** zbierz M kroków kalibracyjnych historii; oblicz „nonconformity scores” $q_1..q_M$ dla wyjść polityki
- **W czasie online:** dla bieżącego kroku oblicz q_t ; jeśli $q_t > \text{kwantyl}_{\{1-\alpha\}}$ z kalibracji $\rightarrow$ ostrzeżenie
- **Gwarancja:** przy prawdziwym rozkładzie danych, fałszywy alarm z częstotliwością $\leq \alpha$ (np. 5%)
- **Zastosowanie:** certyfikacja ze polityka RL NIE narusza ZMP lub stref bezpieczeństwa z $P > 1 - \alpha$
- **Implementacja:** biblioteka MAPIE (Python); integracja jako ROS 2 node z dostępem do historii rollout
- **Odniesienie:** Angelopoulos & Bates (2022) "Conformal Risk Control"; Lindemann et al. (2023) dla robotów

03

SEKCJA

Testowanie i walidacja

HIL, SIL, PIL, scenariusze testowe, benchmarki, metryki



Poziomy testowania: SIL, HIL, PIL

SIL: Software in the Loop

- **Kod sterowania** uruchamiany w symulatorze (Gazebo, Isaac)
- **Brak fizycznego sprzętu**; wszystko wirtualne
- **Szybkie iteracje**: 1000 scenariuszy / godzinę
- Sprawdza logikę, planistę, monitor bezpieczeństwa
- **Ograniczenie**: nie wykrywa błędów sprzętu, ani opóźnień sieci
- **Narzędzia**: pytest + Gazebo, CARLA, AirSim

HIL: Hardware in the Loop

- **Prawdziwy kontroler**, aktuatory lub sterownik wpięty w symulator
- **Symulowane środowisko + realny czas reakcji** mikrokontrolera
- Wykrywa problemy z latencją komunikacji, przerwaniami
- **Koszt**: wymaga symulatora czasu rzeczywistego (dSPACE, NI)
- **Typowy krok**: po SIL, przed testami na pełnym robocie
- Narzędzia: Speedgoat, Opal-RT, NI VeriStand

PIL: Processor in the Loop

- **Docelowy procesor** (Jetson) wpięty w pętlę symulacyjną
- Kod skompilowany na target hardware; sprawdza czas rzeczywisty
- Wykrywa przepełnienia bufora, opóźnienia rzeczywiste
- Pomiedzy HIL, a pełnym testem na robocie
- Kluczowy dla weryfikacji TorchScript / ONNX polityki RL
- **Narzędzia**: ROS 2 bag replay + Jetson + symulator w chmurze

Scenariusze testowe dla nawigacji społecznej

- **TC-01 Prosta trasa 10 m:** brak przeszkod, pomiar RMSE, czasu dotarcia, zużycia energii; 10 powtórzeń
- **TC-02 Statyczna przeszkoda:** skrzynka 0.5×0.5 m w połowie trasy; robot musi ją ominąć bez wchodzenia w strefę 0.3 m
- **TC-03 Dynamiczna przeszkoda:** pieszy przecinający trasę z prędkością 1.2 m/s; metryka: TTC > 1 s w każdej chwili
- **TC-04 Naruszenie strefy osobistej:** aktor zbliży się do robota < 0.5 m; robot musi się zatrzymać w < 300 ms
- **TC-05 Pchnięcie boczne 20 Ns:** robot musi odzyskać równowagę w < 3 s; brak upadku w 95% prób
- **TC-06 Utrata IMU (30 s):** robot musi przejść do trybu degradacji i zatrzymać się bezpiecznie w < 1 s
- **TC-07 Długa misja 30 min:** patrol po budynku; metryki: liczba incydentów, dryft SLAM, zużycie akumulatora

Automatyczny benchmark nawigacji (szkielet pytest)

```
import pytest, time
import numpy as np

# Fikstura: uruchom symulator i robot driver przed testem
@pytest.fixture(scope="session")
def robot_env():
    env = SimEnv(config="corridor_5pedes.yaml") # 5 pieszych
    yield env
    env.close()

def test_tc03_dynamic_obstacle(robot_env):
    """TC-03: robot nie narusza  $TTC < 1$  s z pieszym."""
    env = robot_env
    obs = env.reset(scenario="cross_walker")
    min_ttc = float("inf")
    for _ in range(500): # 500 kroków po 100 ms = 50 s
        action = env.policy(obs)
        obs, _, done, info = env.step(action)
        min_ttc = min(min_ttc, info["ttc"])
        if done: break
    assert min_ttc >= 1.0, f"Naruszenie TTC: {min_ttc:.2f} s"
    assert env.info["success"], "Robot nie dotarł do celu"
```



Opis: TTC = Time-To-Collision; test uruchamiany w CI/CD po każdym merge do branch main

Metryki jakości nawigacji: pełna lista

- **Nawigacja geometryczna:** RMSE pozycji [m], sukces misji [%], czas dotarcia [s], zużycie energii [Wh]
- **Bezpieczeństwo:** liczba kolizji, minimum TTC [s], liczba naruszeń strefy bezpiecznej, max siłą kontaktu [N]
- **Stabilność:** margines ZMP [m], liczba korekt kroku, max odchylenie roll/pitch [deg]
- **Nawigacja społeczna:** Intrusion into Personal Space (IPS) [%], Social Space Violations (SSV) [#], Path Irregularity (PI)
- **Jakość lokalizacji:** RMSE pozycji vs. ground truth (Vicon), błąd yaw [deg], dryft SLAM [m/km]
- **Wydajność obliczeniowa:** latencja planisty [ms], obciążenie CPU/GPU [%], temperatura Jetson [°C]
- **Subiektywne:** Perceived Safety Rating (PSR, 1-7), Comfort Score (CS, 1-7), NASA-TLX dla operatora

04

SEKCJA

Wdrażanie i utrzymanie

Architektura floty, OTA updates, monitoring operacyjny, etyka



Cykl zycia systemu nawigacji humanoidalnej

1

Projektowanie

Analiza wymagań (Safety, Functional), HAZOP/FMEA, wybór stosu technologicznego i architektury systemu

2

Integracja i SIL

Budowa systemu, testy jednostkowe, integracyjne i SIL w Gazebo/Isaac; CI/CD pipeline na GitHub Actions

3

HIL i PIL

Testy na docelowym hardware (Jetson); weryfikacja latencji; testy regresji przy każdym release RC

4

Certyfikacja

Audyt przez jednostkę notyfikowaną; analiza FMEA; dokumentacja CE; testy wytrzymałościowe 1000 h

Architektura floty i zarządzanie flotą robotów

Infrastruktura centralna (cloud)

- **Fleet Manager:** rejestr robotów, przypisanie zadań, priorytetyzacja misji
- **OTA Update Server:** dystrybucja nowych polityk RL i modeli VLM przez HTTPS TLS 1.3
- **Digital Twin:** wirtualna kopia każdego robota; symulacja nowych polityk przed wdrożeniem
- **Centralne logowanie:** ELK Stack / Grafana; zbieranie rosbag2 fragmentów przy incydentach
- **A/B testing polityk:** 10% floty testuje nowszą politykę; rollback automatyczny gdy metryki pogorszone

Architektura lokalna (robot)

- **Modul OTA:** pobiera update poza godzinami pracy; walidacja sumy SHA-256 przed instalacją
- **Dual-bank OS:** aktywny bank + standby; rollback jednym poleceniem przy awarii update
- **Lokalny monitoring:** edge analytics na Jetson; alert jeśli anomalia wykryta bez połączenia z chmurą
- **Security:** mTLS między robotem a serwerem; certyfikaty X.509 rotowane co 90 dni
- **GDPR:** dane biometryczne (twarze) przetwarzane tylko lokalnie; nie wysłane do chmury

OTA Update: weryfikacja i bezpieczna instalacja polityki

```
import hashlib, os, shutil, torch

def verify_and_install_policy(url: str, expected_sha256: str,
                             policy_path="/opt/robot/policy.pt") -> bool:
    """Pobiera, weryfikuje i instaluje nowa polityke RL przez OTA."""
    import urllib.request
    tmp_path = "/tmp/policy_candidate.pt"
    # 1. Pobierz plik z serwera OTA (TLS)
    urllib.request.urlretrieve(url, tmp_path)
    # 2. Weryfikacja sumy SHA-256
    sha = hashlib.sha256(open(tmp_path, "rb").read()).hexdigest()
    if sha != expected_sha256:
        os.remove(tmp_path); return False # kompromitacja pliku
    # 3. Walidacja modelu (forward pass z testowym wejściem)
    model = torch.jit.load(tmp_path).eval()
    test_obs = torch.zeros(1, 18)
    action = model(test_obs) # sprawdz ze nie rzuca wyjątku
    assert action.shape == (1, 2), "Niepoprawny kształt wyjścia!"
    # 4. Atomiczna podmiana (backup starego)
    shutil.copy2(policy_path, policy_path + ".bak")
    shutil.move(tmp_path, policy_path) # atomiczne rename
    return True
```

Opis: Atomiczna zmiana nazwy zapobiega częściowej instalacji; SHA-256 chroni przed podmianą pliku

Monitoring operacyjny: co metryzować w produkcji

- **KPI misji:** sukces [%], średni czas wykonania [s], liczba interwencji operatora dziennie
- **Bezpieczeństwo:** incydenty/miesiąc (kolizje, zatrzymania awaryjne, upadki, skargi użytkowników)
- **Wydajność:** zużycie energii na misje [Wh], czas lądowania, czas dostępności (uptime [%])
- **Zdrowie systemu:** temperatura aktuatorów, błędy enkoderów, zużycie akumulatora (state-of-health)
- **Jakość nawigacji:** RMSE lokalizacji (spot-check z zewnętrznym systemem), dryft mapy na koniec dnia
- **ML drift:** porównanie rozkładu obserwacji online vs. dane treningowe (PSI score); alert przy > 0.2
- **Narzędzia:** Grafana + InfluxDB (metryki); ELK (logi); PagerDuty (alerty 24/7); DataDog (ML monitoring)

Aspekty etyczne i społeczne wdrażania humanoidów

- **Prywatność:** humanoid z kamerami to de facto urządzenie nadzoru; wymaga polityki RODO i zgody użytkowników
- **Transparentność:** użytkownik ma prawo wiedzieć że jest obserwowany przez robota i co system robi z danymi
- **Biasy algorytmów:** modele ML nauczone na niereprezentatywnych danych mogą gorzej rozpoznawać pewne grupy osób
- **Przyszłość pracy:** humanoid wykonujący zadania wcześniej ludzkie - kontekst społeczny i odpowiedzialność pracodawcy
- **Odpowiedzialność:** kto odpowiada za szkody - producent, operator czy użytkownik? Ramy prawne UE AI Act (2024)
- **EU AI Act:** roboty nawigujące w przestrzeni publicznej zakwalifikowane jako systemy wysokiego ryzyka (Annex III)
- **Zasada:** technika może, ale to prawo i etyka decydują czy i jak. To inżynier odpowiada więc za świadome projektowanie

PODSUMOWANIE

Nawigacja Robotów Humanoidalnych

W1

Percepcja i mapowanie

W2

Lokalizacja i fuzja

W3

Planowanie ruchu

W4

Chod i stabilność

W5

VLM / VLA

W6

World-Model Action

W7

Nawigacja społeczna

W8

Reinforcement Learning

W9

Bezpieczeństwo i wdrożenie

***Pełna pętla:** Sensory -> Percepcja -> Lokalizacja -> Planowanie -> WMA/RL/VLA -> Sterowanie -> Aktuatory -> Sensory*

PODSUMOWANIE

01

ISO 13482, SIL/PLe i FMEA to niezbędne ramy prawne i inżynierskie przed wdrożeniem humanoida wśród ludzi.

02

Architektura bezpieczeństwa musi być wielowarstwowa: hardware PLe + RTOS watchdog + runtime monitor + conformal prediction.

03

Testowanie SIL/HIL/PIL i automatyczne benchmarki (pytest + Gazebo) pozwalają wykryć regresje przed wdrożeniem na robocie.

04

OTA updates, dual-bank OS i monitoring ML drift są kluczowe dla długookresowej niezawodności floty humanoidów.

05

Aspekty etyczne (prywatność, biasy, EU AI Act) są równoważne z aspektami technicznymi - inżynier ponosi współodpowiedzialność.

Zadanie projektowe i pytania problemowe

1. Dla jakiego poziomu SIL powinien być certyfikowany czujnik F/T w stopie humanoida pracującego w szpitalu? Uzasadnij opierając się na ISO 13482 i IEC 61508.
2. Zaprojektuj scenariusz testowy TC-08 dla robota wchodzącego do windy: jakie warunki brzegowe należy sprawdzić, jakie metryki zmierzyć i ile powtórzeń jest wymagane dla statystycznej pewności?
3. Czym różni się SIL (IEC 61508) od PLe (EN ISO 13849) i dlaczego system nawigacyjny może wymagać jednocześnie obu certyfikatów?

Zadanie: napisz prosty Safety Monitor (Kod 2) i dodaj test pytest sprawdzający że monitor poprawnie ustawia */safety_ok = False* gdy *predkosc > v_max* lub *temperatura > temp_warn*. Uruchom test z symulowanymi danymi bez prawdziwego robota.

Bonus: przeanalizuj jeden publicznie dostępny raport incydentu z robotem usługowym (np. Knightscope, Softbank Pepper) i wskazać które z omawianych warstw bezpieczeństwa zawiodły lub były nieobecne.